Technical Requirements Document

AICTE Approval Workflow Portal

Version: 1.0 · **Status:** Prototype specification · **Date:** 22 October 2026

1. Recommended stack

- **Frontend:** React + TypeScript, Vite or an equivalent maintained framework.
- **UI:** Tailwind CSS plus accessible reusable components; Lucide icons.
- **Backend/data:** PostgreSQL via Supabase or a comparable managed backend.
- **Authentication:** Supabase Auth or an equivalent provider; server-verified sessions.
- **Files:** private object storage with signed, short-lived access URLs and server-side authorisation.
- **AI/OCR:** provider-independent service interface for OCR, document classification and optional language-model analysis.
- **Testing:** unit tests for workflow/deadline rules; integration tests for permissions and uploads; end-to-end tests for the principal user journeys.
- **Deployment:** environment-based configuration; separate development/demo/production settings.

The stack is a recommendation, not a requirement from AICTE. Do not place service-role secrets or model API keys in frontend code.

2. Application modules

1. Authentication and role/organisation context.
2. Institution and user management.
3. Application intake and document checklist.
4. Workflow configuration and transition engine.
5. Case review and deficiency resolution.
6. EVC queue and schedule tracking.
7. Undertaking/compliance obligation management.
8. Document validity and renewal management.
9. Notification, escalation and scheduled-job service.
10. Dashboard analytics and reports.
11. Audit log.
12. AI/OCR pre-submission checker.
13. Administration and configuration.

3. Core data entities

- `users`: identity, active state, role assignment references.
- `organisations`: institution identifiers, name, region and permitted parent/affiliation relationships.
- `memberships`: user-to-organisation/region scope and permissions.
- `applications`: reference ID, institution, application type, current status/stage, route version, timestamps and assigned owner.
- `workflow_definitions`: versioned application route and transition rules.

- `workflow_events`: transition, actor, reason, timestamp and metadata.
- `documents`: application/obligation association, storage key, file metadata, version, uploader and scan status.
- `document_checks`: check run, rule version, extracted fields, warnings, confidence and outcome.
- `deficiencies`: requirement/source, description, responsible party, due date, status and resolution.
- `evc_records`: requirement/readiness, schedule, completion and authorised outcome.
- `obligations`: condition/undertaking type, due date, evidence requirements and status.
- `document_validities`: applicable document type, start/expiry dates, renewal state and verification state.
- `notifications`: recipient, type, due/created/sent/acknowledged timestamps and delivery status.
- `escalations`: policy, trigger, recipient, status and resolution.
- `audit_logs`: actor, action, entity, timestamp, reason and safe metadata.
- `workflow_rule_versions`: source reference, effective dates, authoriser and change summary.

Use UUIDs or equivalent non-guessable identifiers internally. Human-readable application references can be separate display IDs.

4. API/service requirements

Expose server-authorised operations for:

- Create/update/submit application.
- Retrieve applications filtered by permission scope.
- Validate and execute workflow transitions.
- Create deficiency and submit correction.
- Run pre-submission checks and retrieve results.
- Create/complete EVC scheduling tasks.
- Create/submit/verify compliance evidence.
- Record document validity and renewal.
- Generate notifications and escalations.
- Retrieve dashboard metrics and reports.
- Manage versioned workflow configuration.
- Retrieve audit events.

Every mutating operation must verify the user's role, scope, current record state and permitted action on the server. Frontend-only hiding is not security.

5. Workflow engine

- Routes must be data/configuration-driven, not scattered hardcoded conditionals.
- Each transition defines allowed source stage, destination stage, required prerequisites, authorised roles, required fields/evidence and whether a reason is mandatory.
- Support conditional paths such as scrutiny-only, scrutiny + EVC, EVC + SAC/SHC and other routes only when configured from an authorised source.
- Support authorised return/revert and re-review loops.
- Reject invalid transitions server-side and record the rejection where appropriate.
- Version workflow definitions. Existing cases must retain an explainable route version or be migrated through an authorised process.

- No automated final approval unless an explicit lawful rule and competent authority action are represented; default is human decision.

6. Deadlines and escalations

- Store due dates with timezone and rule/source reference.
- Separate statutory/prescribed deadlines from internal target dates.
- Use a scheduled backend job/queue; do not rely on a browser tab being open.
- Make reminders idempotent so repeated job runs do not create duplicate notifications.
- Escalation policies specify trigger, delay, recipient role, channel and maximum repeat frequency.
- Keep a log of scheduled, sent, failed, escalated and closed events.
- Changing a rule must not silently rewrite historic deadlines; apply changes only under an authorised versioned policy.

7. Document pre-check pipeline

1. Upload to private quarantine/staging storage.
2. Validate file type/size and scan for malicious content using available infrastructure.
3. Extract text/OCR and page-level extraction status.
4. Classify document type if enabled.
5. Run deterministic field/date/completeness checks using a versioned checklist.
6. Optionally run AI-assisted comparison/explanation against extracted text and configured rules.
7. Store findings with evidence spans/page references, rule IDs, confidence and limitations.
8. Present findings to the institution; allow replacement and rerun.
9. Preserve version history and link official review findings separately.

Never treat AI output as proof of forgery or an official compliance decision. If an authorised external verification source is unavailable, mark verification as pending/source unavailable.

8. Security and privacy

- Enforce least privilege and organisation/region-level row access.
- Use database row-level security where supported, plus server-side checks.
- Store files in private buckets; do not expose permanent public URLs.
- Validate uploads and prevent path traversal, unsafe file execution and oversized files.
- Keep secrets in server environment variables/secret manager.
- Apply rate limiting to login and upload endpoints.
- Audit sensitive administrative changes.
- Minimise personal data in demo datasets; use synthetic data where possible.
- Define retention, deletion, backup and incident-response practices before production.
- Avoid sending confidential documents to third-party AI services unless authorised and contractually/privacy compliant.

9. Reliability and accessibility

- Responsive support for desktop and tablet; mobile usability for essential actions.
- Keyboard-operable controls, labelled inputs, visible focus and WCAG AA contrast targets.
- Honour `prefers-reduced-motion`.

- Loading, empty, retry, partial-failure and error states for every network-driven view.
- Pagination/server-side filtering for large tables.
- Store timestamps consistently (UTC recommended) and render in a declared local timezone.
- Use structured logs and health checks for backend jobs.

10. Environment and configuration

- `.env.example` contains names only, never secrets.
- Distinct demo and production environments.
- Synthetic seeded users and records for the demo.
- Feature flags for optional AI, external integrations and university access.
- No claim of live AICTE integration unless a real authorised connection is implemented and tested.